

RepMax

Sistema de Log Book de Treino de Academia

Documentação de Análise, Requisitos e Arquitetura do Sistema

Artur Pereira e Zadoque Carneiro

Disciplina: POODev

Sumário

1	Introdução	3
1.1	Problema de Negócio	3
1.2	Proposta do Trabalho	3
2	StakeHolders	4
2.1	Aluno	4
2.2	ADMIN	4
3	Descrição Técnica do Sistema	5
3.1	API RESTful	5
3.2	Interface WEB:	5
3.3	Bloco: Treino Individual	6
3.4	Bloco: Acompanhamento por Personal Trainer	6
4	Estudo de Viabilidade	6
4.1	Viabilidade Técnica	6
4.2	Viabilidade Operacional	6
4.3	Viabilidade Econômica e Orçamento	7
4.4	Análise de Riscos	7
5	Requisitos do Sistema	8
5.1	Requisitos Funcionais	8
5.2	Requisitos Não Funcionais	9
6	Diagramas de Fluxo de Dados — DFD	10
6.1	DFD de Contexto	10
6.2	DFD de Registro de Treino	10
7	Metodologia de Desenvolvimento	10
7.1	Metodologia Ágil	10
7.2	Cronograma Estimado em Sprints	10

8	Modelagem UML — Diagrama de Classes	11
8.1	Classe Usuario	11
8.2	Classe Perfil	12
8.3	Classe Exercício	12
8.4	Classe Treino	13
8.5	Classe SerieExecutada	13
8.6	Classe RotinaTreino	14
8.7	Classe PlanoTreino	14
8.8	Classe FeedbackTreino	14
9	Relacionamentos e Cardinalidades	14
10	Modelo Entidade-Relacionamento — ER (Resumo das Tabelas)	15
11	Casos de Uso	16
11.1	Casos de Uso do Aluno	16
11.2	Casos de Uso do Administrador	16
12	Arquitetura da Aplicação	16
13	Endpoints da API (Principais)	17
14	Regras de Negócio	17
14.1	Não negatividade da carga	17
14.2	Enumeração de bi-set, tri-set e monster-set	17
14.3	Diminuição da carga em drop set	18
14.4	Fórmula do cálculo de 1RM e restrição a quantidade de repetições	18
14.5	Um aluno não pode visualizar treinos e histórico de outro aluno	18
14.6	Transparência no propósito de feedbacks do Rep Max	18
15	Segurança e Privacidade	18
16	Conclusão	19
17	Referências	19

1 Introdução

O crescimento contínuo da prática de atividades físicas no Brasil, evidenciado pelos dados da Sociedade Brasileira de Endocrinologia e Metabologia (SBEM)¹, impulsiona a demanda por soluções digitais capazes de organizar e acompanhar a evolução de treinos de forma estruturada. Assim, o **RepMax** surge como uma plataforma web voltada ao registro de treinos de musculação, à visualização de progresso, somando mais tecnologia ao meio fitness.

1.1 Problema de Negócio

Observam-se, entre praticantes de musculação, as seguintes dificuldades recorrentes:

- Ausência de um histórico organizado de séries, repetições e cargas utilizadas em cada treino;
- Dificuldade de identificar se um exercício está em progressão, estagnação ou regressão de desempenho;
- Falta de suporte para técnicas de intensificação de treino (bi-set, tri-set, monster set e drop set), que hoje são anotadas de forma manual e pouco estruturada;
- Dúvidas frequentes sobre a execução correta de determinados exercícios, sem uma referência disponível no momento do treino;

1.2 Proposta do Trabalho

Embora esse projeto tenha como objetivo principal resolver o problema de negócio e agregar valor para o cliente, há também o propósito de aprendizado de conceitos. Por isso, esse trabalho propõe o desenvolvimento de uma API RESTful para um sistema de log book de treino, denominado **RepMax**. E uma interface WEB usando: React + Typescript + Tailwind CSS + React Router (**Data Mode**) + Vite + fetch() para interagir com ela (Mais abaixo tem a justificativa para o uso de cada parte da interface). A solução tem como objetivo permitir que o usuário registre seus treinos de forma estruturada, acompanhe sua progressão por meio de indicadores, tais como: a repetição máxima estimada (1RM) e técnicas avançadas de intensidade.

¹SOCIEDADE BRASILEIRA DE ENDOCRINOLOGIA E METABOLOGIA (SBEM). *Dados sobre a prática de atividade física no Brasil*. [link oficial e ano da pesquisa — campo editável, a ser preenchido com a fonte efetivamente consultada].

2 Stakeholders

2.1 Aluno

Um aluno é um usuário cadastrado no sistema por um admin que não possui vínculo com um personal. Ele pode:

- Fazer login no sistema com email e senha;
- Trocar sua senha com código enviado por email;
- Criar uma nova rotina de treino;
- Visualizar suas próprias rotinas de treino;
- Editar uma rotina de treino;
 - Substituir um exercício por outro;
 - Excluir um exercício;
 - Adicionar um exercício;
 - Editar um exercício;
 - * Alterar o número de séries;
- Excluir uma rotina de treino;
- Começar uma rotina de treino;
- Pausar uma rotina de treino;
- Encerrar uma rotina de treino;
- Visualizar o próprio histórico em calendário;

2.2 ADMIN

O usuário admin é o usuário que consegue gerenciar os outros usuários. Ele pode:

- Pesquisar por usuários no sistema;
- Adicionar usuário como aluno no sistema;
- Adicionar usuário como Admin no sistema;
- Remover um usuário aluno do sistema;
- Enviar código para o email de um usuário para ele poder trocar sua senha;
- Cadastrar Exercícios no sistema;
- Editar Exercícios no sistema;

3 Descrição Técnica do Sistema

3.1 API RESTful

Ela será responsável por gerenciar o histórico de treino de usuários. De forma geral, o sistema deverá permitir:

- Cadastrar e autenticar usuários (alunos e admins);
- Cadastrar exercícios, com grupo muscular e instruções de execução;
- Registrar treinos, associando exercícios, séries, repetições e cargas;
- Registrar séries em formatos de intensificação: bi-set, tri-set, monster set e drop set;
- Calcular a repetição máxima estimada (1RM) a partir de carga e repetições;
- Consultar o histórico e a evolução de desempenho por exercício;
- (Versão 2) Vincular aluno e personal trainer, permitindo que o personal monte treinos, o aluno registre feedback e ambos se comuniquem dentro do próprio sistema.

3.2 Interface WEB:

O conjunto de tecnologia (stack) foi escolhido para viabilizar o aprendizado de conceitos como:

- rotas;
- cache;
- segurança;
- server-side;
- client-side.

Cada parte da interface tem seu propósito:

- Vite -> Ferramenta responsável pelo ambiente dev e processo de build do front-end
- React -> Biblioteca de UI, para construção e navegação na interface.
- Tailwind CSS -> Para estilização.
- React Router no modo Data -> Não faz mágica sozinho, mas traz ferramentas para eu aprender e usar como Rotas, Loader, Action e Estados de navegação.
- fetch() -> Para fazer requisição à API RESTful.

Esse conjunto permite que tenhamos um equilíbrio entre conceitos a aprender e aplicar e prazo de entrega.

3.3 Bloco: Treino Individual

Concentra as funcionalidades de uso autônomo do aluno: cadastro de exercícios, registro de séries (incluindo as técnicas de intensidade), cálculo de 1RM, histórico e rotinas.

3.4 Bloco: Acompanhamento por Personal Trainer

Reúne as funcionalidades da segunda versão do sistema: vínculo entre aluno e personal, montagem de planos de treino pelo profissional, feedback do aluno e comunicação entre as partes.

4 Estudo de Viabilidade

4.1 Viabilidade Técnica

O sistema é tecnicamente viável em razão do uso de tecnologias consolidadas e amplamente adotadas no mercado de desenvolvimento de software:

- Java, como linguagem de programação orientada a objetos;
- Spring Boot, para a criação simplificada da API;
- Spring Web, para a exposição dos endpoints HTTP;
- Spring Security, para segurança (JWT, Argon2);
- Spring Data JPA e Hibernate, para a camada de persistência de dados (ORM);
- PostgreSQL, como sistema gerenciador de banco de dados relacional;
- Maven, para o gerenciamento de dependências do projeto;
- Docker e Docker Compose, para padronizar o ambiente de execução.

A adoção da arquitetura em camadas (Controller, Service, Repository, Entity/DTO), detalhada na Seção 11, facilita a manutenção, os testes e a evolução futura do sistema, especialmente relevante dado o caráter incremental do projeto: a intenção declarada é iniciar com um escopo simples e evoluir gradualmente até um sistema completo.

4.2 Viabilidade Operacional

Do ponto de vista operacional, o sistema RepMax pode auxiliar diretamente os seguintes perfis de usuário:

- **Alunos**, que passam a ter um histórico estruturado de treino e indicadores de progressão;
- **Personal trainers**, que passam a montar e acompanhar treinos de seus alunos diretamente pela plataforma;
- **Administradores**, responsáveis pela gestão geral de usuários e exercícios cadastrados no sistema.

4.3 Viabilidade Econômica e Orçamento

Por se tratar de um projeto acadêmico desenvolvido por dois alunos, não há custos reais de mão de obra envolvidos. Apresenta-se a seguir uma estimativa ilustrativa de custo baseada no tempo de desenvolvimento.

Item	Descrição	Valor
Hora técnica	Valor médio de referência para desenvolvedor júnior/estudante	R\$ 75,00
Horas diárias	2 a 3 horas em dias úteis (média de 2,5h), por desenvolvedor	—
Semanas estimadas	12 semanas	—
Total de horas	$2,5h \times 5 \text{ dias} \times 12 \text{ semanas} \times 2$ desenvolvedores	$\approx 300 \text{ horas}$
Custo total estimado	$300 \text{ horas} \times R\$ 75,00$	$\approx R\$ 22.500,00$

Não há custos com licenças de software (Java, Spring Boot, PostgreSQL e Docker são gratuitos e de código aberto), banco de dados ou hospedagem, uma vez que o desenvolvimento e os testes ocorrem em ambiente local/containerizado.

4.4 Análise de Riscos

Risco	Impacto	Prob.	Estratégia de mitigação
Registro incorreto de séries combinadas (bi-set/tri-set/monster set)	Alto	Média	Validar na camada de Service a quantidade mínima de exercícios exigida por tipo de técnica.
Cálculo impreciso da carga máxima estimada	Médio	Baixa	Cobrir a fórmula de 1RM com testes unitários e restringir seu uso a faixas de repetição confiáveis (até 10-12 reps).
Falhas no vínculo aluno-personal	Alto	Média	Exigir aceite mútuo e validar unicidade do vínculo ativo antes de novas solicitações.
Exposição indevida de dados entre alunos	Alto	Média	Utilizar DTOs e controle de acesso por perfil (RBAC), garantindo que um usuário só acesse seus próprios dados ou os de alunos vinculados.
Atraso no cronograma devido à carga horária limitada	Médio	Alta	Adotar metodologia ágil com priorização das funcionalidades essenciais (Fase 1) antes das avançadas (Fases 2 e 3).
Dificuldade de aprendizado de tecnologias novas (WebSocket, Docker)	Médio	Alta	Reservar tempo para estudo dirigido e priorizar documentação oficial das tecnologias.

5 Requisitos do Sistema

5.1 Requisitos Funcionais

ID	Descrição	Prior.	Status
RF01	Permitir o cadastro de um novo usuário.	Alta	Não implementado
RF02	Permitir que o usuário realize login por e-mail e senha.	Alta	Não implementado
RF03	Permitir o CRUD de exercícios (nome, grupo muscular, descrição).	Alta	Não implementado
RF04	Permitir o registro de uma sessão de treino, com data, exercícios, séries, repetições e carga.	Alta	Não implementado
RF05	Permitir a consulta do histórico de treinos do usuário.	Alta	Não implementado
RF06	Calcular automaticamente a repetição máxima estimada (1RM), a partir de carga e repetições, usando a fórmula de referência de Epley.	Média	Não implementado
RF07	Exibir a evolução de carga e volume de um exercício ao longo do tempo.	Média	Não implementado
RF08	Permitir a criação de rotinas de treino reutilizáveis (ex.: Treino A, B, C).	Média	Não implementado
RF09	Permitir o registro de séries em bi-set (dois exercícios em sequência, sem descanso).	Média	Não implementado
RF10	Permitir o registro de séries em tri-set (três exercícios em sequência, sem descanso).	Média	Não implementado
RF11	Permitir o registro de séries em monster set (quatro ou mais exercícios em sequência, sem descanso).	Média	Não implementado
RF12	Permitir o registro de séries em drop set (redução progressiva de carga no mesmo exercício, sem descanso).	Média	Não implementado
RF13	Exibir, para cada exercício, uma demonstração de execução correta (texto, imagem ou vídeo).	Alta	Não implementado
RF14	Permitir a edição e exclusão de treinos e exercícios cadastrados.	Alta	Não implementado

ID	Descrição	Prior.	Status
RF15	Permitir a filtragem do histórico por período, exercício ou grupo muscular.	Média	Não implementado

5.2 Requisitos Não Funcionais

ID	Descrição	Status
RNF01	API RESTful, seguindo boas práticas de verbos e códigos de status HTTP.	Não implementado
RNF02	Persistência dos dados em PostgreSQL via Spring Data JPA.	Não implementado
RNF03	Arquitetura organizada em camadas (Controller, Service, Repository, Entity/DTO).	Não implementado
RNF04	Uso de DTOs para desacoplar entidades de persistência das requisições/respostas.	Não implementado
RNF05	Documentação de API via OpenAPI/Swagger.	Não implementado
RNF06	Senhas armazenadas de forma criptografada (Argon2).	Não implementado
RNF07	Tratamento centralizado de exceções, com respostas de erro padronizadas.	Não implementado
RNF08	Tempo de resposta inferior a 1s nas operações comuns de consulta e registro.	Não implementado
RNF09	Sistema executável em contêineres Docker, com configuração via Docker Compose.	Não implementado
RNF10	Código-fonte versionado em Git, com commits organizados por funcionalidade.	Não implementado
RNF11	Testes unitários e de integração para as regras de negócio críticas (cálculo de 1RM, técnicas combinadas).	Não implementado
RNF12	Validar dados obrigatórios antes da persistência (carga não negativa, repetições mínimas etc.).	Não implementado

6 Diagramas de Fluxo de Dados — DFD

6.1 DFD de Contexto

O sistema RepMax é representado como um processo único, interagindo com dois atores externos (Aluno e Administrador) e com o banco de dados PostgreSQL. Os principais fluxos são: dados de cadastro e treino (Aluno → Sistema), planos de treino montados (Personal → Sistema → Aluno), feedback e mensagens (Aluno ↔ Sistema ↔ Personal), operações de leitura/gravação junto ao banco de dados, e respostas da API para todos os atores.

6.2 DFD de Registro de Treino

1. O aluno envia os dados do treino (exercícios, séries, repetições, carga, tipo de técnica).
2. A API recebe e valida as informações.
3. A camada Service aplica as regras de negócio (ex.: validar quantidade mínima de exercícios em um monster set).
4. A camada Repository persiste os dados.
5. O banco de dados PostgreSQL confirma a operação.
6. A API retorna o treino registrado (TreinoResponseDTO), incluindo o 1RM estimado calculado.

7 Metodologia de Desenvolvimento

7.1 Metodologia Ágil

O desenvolvimento seguirá princípios ágeis, adaptados à realidade de um projeto acadêmico com carga horária diária limitada: desenvolvimento iterativo e incremental, entregas parciais contínuas, e priorização das funcionalidades essenciais antes das complementares — refletindo diretamente o caráter evolutivo do projeto (do simples ao completo).

7.2 Cronograma Estimado em Sprints

Sprint	Semana	Foco principal
Sprint 1	Semana 1–2	Levantamento de requisitos, modelagem UML e modelo ER inicial.
Sprint 2	Semana 3–4	Configuração do projeto Spring Boot, entidades JPA e estrutura em camadas.
Sprint 3	Semana 5–6	CRUD de usuários, exercícios e registro básico de treinos.
Sprint 4	Semana 7–8	Cálculo de 1RM, histórico de progressão e rotinas.

Sprint 5	Semana 9–10	Técnicas de intensidade (bi-set, tri-set, monster set, drop set) e instruções de execução.
Sprint 6	Semana 11–12	Versão 2: vínculo aluno-personal, montagem de planos, feedback e chat.

8 Modelagem UML — Diagrama de Classes

8.1 Classe Usuario

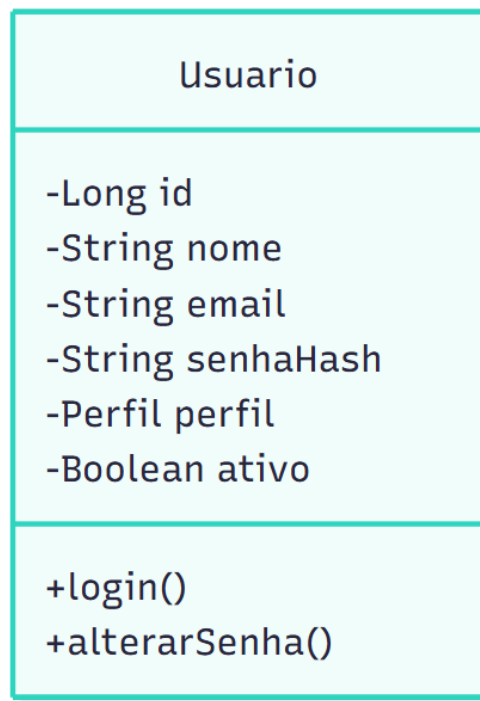
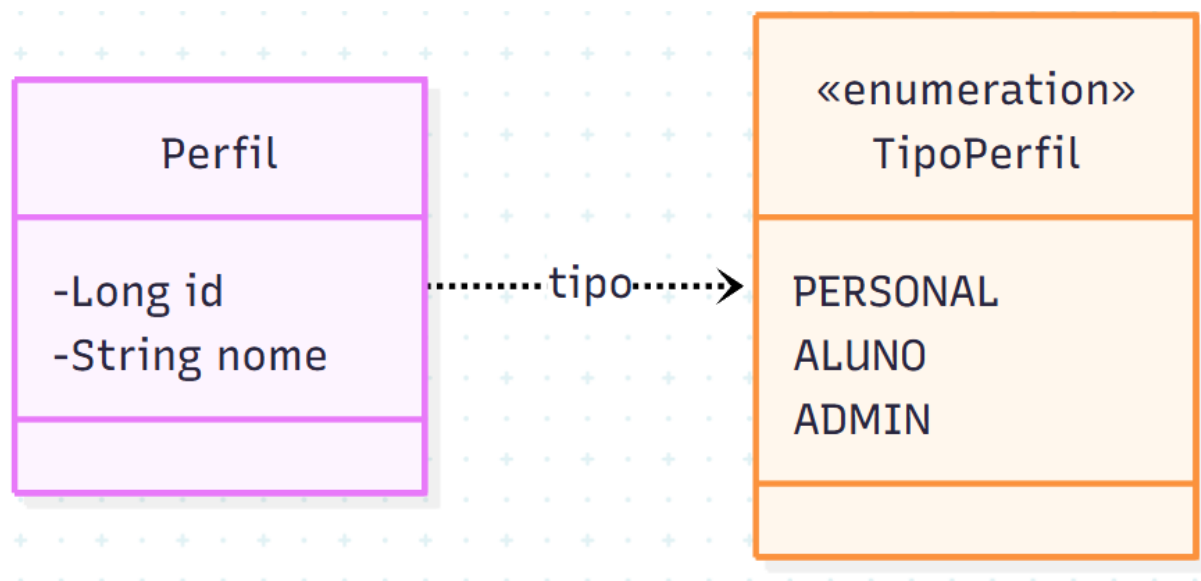
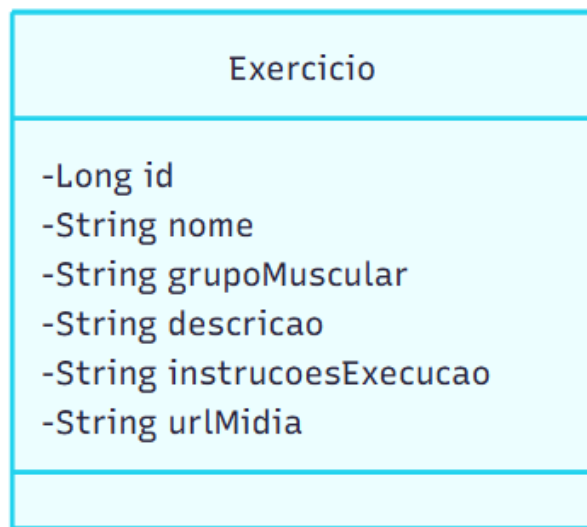


Figura 1: UML usuário

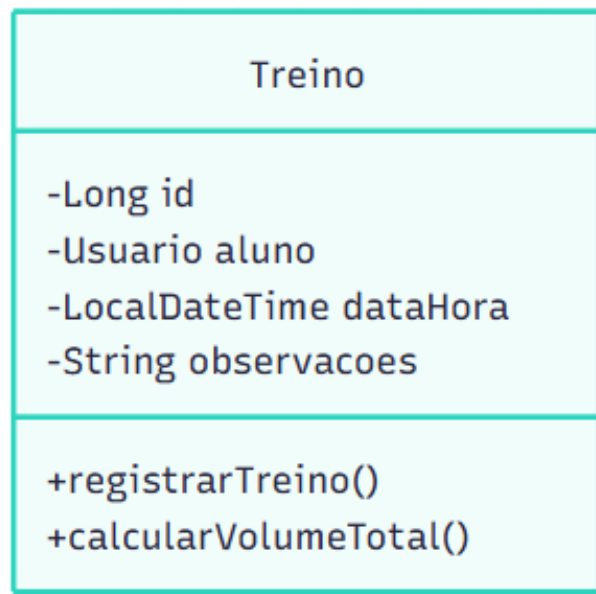
8.2 Classe Perfil



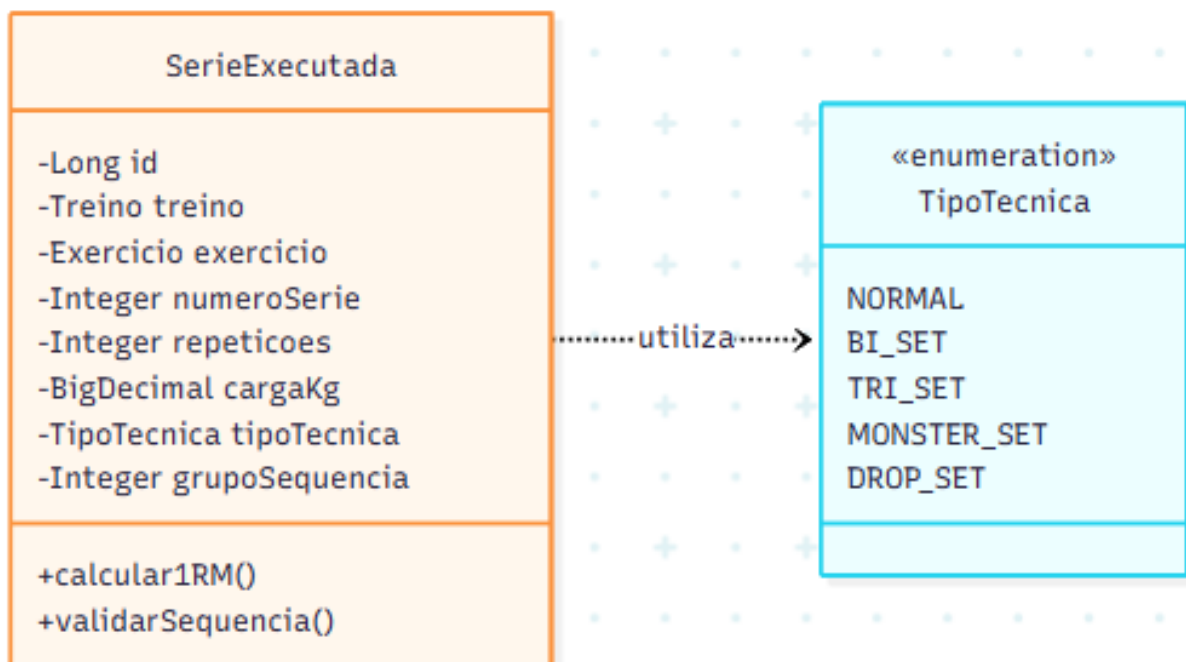
8.3 Classe Exercicio



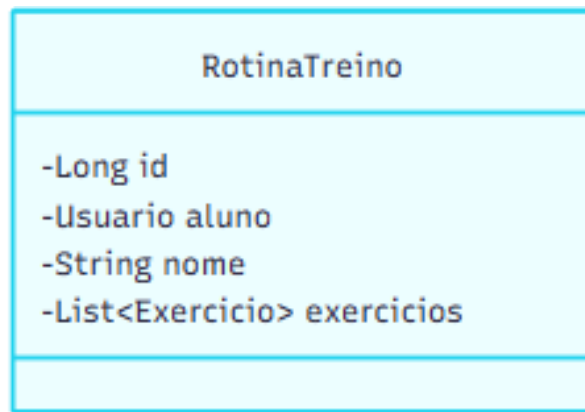
8.4 Classe Treino



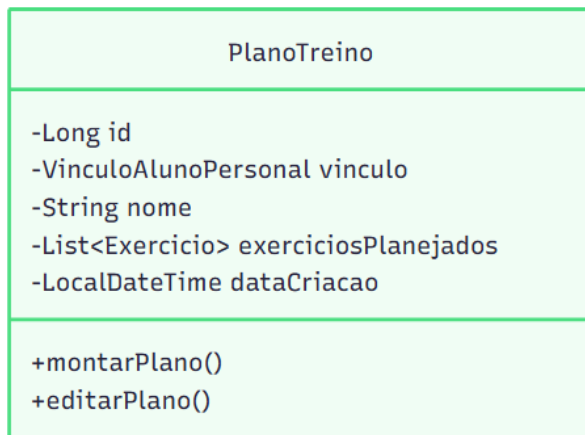
8.5 Classe SerieExecutada



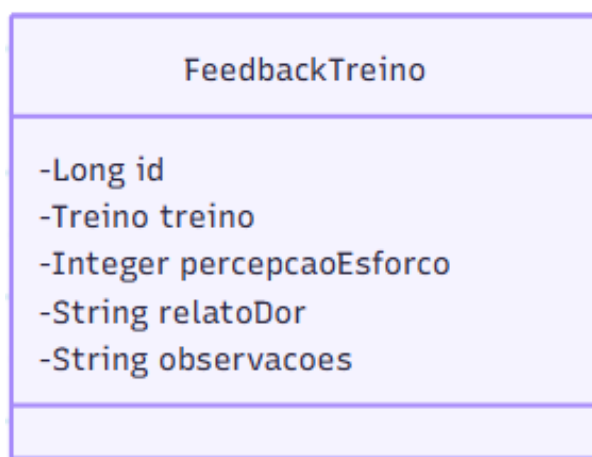
8.6 Classe RotinaTreino



8.7 Classe PlanoTreino



8.8 Classe FeedbackTreino



9 Relacionamentos e Cardinalidades

Classe A	Card.	Classe B	Descrição
Usuario	N:1	Perfil	Cada usuário possui um único perfil de acesso.
Usuario	1:N	Treino	Um aluno pode ter vários treinos registrados.
Treino	1:N	SerieExecutada	Um treino contém várias séries executadas.
Exercicio	1:N	SerieExecutada	Um exercício pode aparecer em várias séries.
Usuario	1:N	RotinaTreino	Um aluno pode criar várias rotinas.
Usuario	1:N	VinculoAluno Personal	Um aluno/personal pode ter vários vínculos.
VinculoAluno Personal	1:N	PlanoTreino	Um vínculo pode gerar vários planos de treino ao longo do tempo.
Treino	1:1	FeedbackTreino	Cada treino pode ter um feedback do aluno associado.

Decisão de modelagem — Técnicas combinadas: bi-set, tri-set e monster set são modelados como múltiplas instâncias de **SerieExecutada** compartilhando o mesmo **grupoSequencia** e **tipoTecnica**, em vez de criar uma entidade separada para cada técnica. Essa decisão simplifica o modelo relacional e permite reaproveitar a mesma tabela para séries normais e combinadas, com a camada de Service validando a quantidade mínima de exercícios exigida por tipo (2 para bi-set, 3 para tri-set, 4+ para monster set).

10 Modelo Entidade-Relacionamento — ER (Resumo das Tabelas)

Tabela	Chave	Principais colunas
USUARIO	PK id	nome, email (UNIQUE), senha_hash, perfil_id (FK), ativo
PERFIL	PK id	nome
EXERCICIO	PK id	nome, grupo_muscular, descricao, instrucoes_execucao, url_midia
TREINO	PK id, FK aluno_id	data_hora, observacoes
SERIE_EXECUTADA	PK id, FK treino_id, FK exercicio_id	numero_serie, repeticoes, carga_kg, tipo_tecnica, grupo_sequencia
ROTINA_TREINO	PK id, FK aluno_id	nome
PLANO_TREINO	PK id, FK vinculo_id	nome, data_criacao
FEEDBACK_TREINO	PK id, FK treino_id	percepcao_esforco, relato_dor, observacoes

11 Casos de Uso

11.1 Casos de Uso do Aluno

UC01 — Autenticar-se: o aluno cria uma conta e realiza login (RF01, RF02).

UC02 — Registrar treino: o aluno registra exercícios, séries, repetições e cargas de uma sessão, podendo indicar técnicas de bi-set, tri-set, monster set ou drop set (RF04, RF09–RF12).

UC03 — Consultar histórico e progressão: o aluno visualiza seus treinos anteriores e a evolução de carga/volume por exercício (RF05, RF07).

UC04 — Consultar instruções de execução: o aluno visualiza como executar corretamente um exercício (RF13).

UC05 — Criar rotina: o aluno organiza exercícios em rotinas reutilizáveis (RF08).

UC08 — Registrar feedback: o aluno relata esforço, dor ou observações após o treino (RFP05).

11.2 Casos de Uso do Administrador

UC14 — Gerenciar exercícios: o administrador cadastra, atualiza ou remove exercícios do catálogo geral (RF03).

UC15 — Gerenciar usuários: o administrador ativa, desativa ou altera permissões de usuários do sistema.

Exemplo detalhado — UC02 (Registrar treino com bi-set):

Objetivo: Registrar uma sessão de treino contendo uma combinação de bi-set.

Pré-condições: O aluno deve estar autenticado; os exercícios envolvidos devem estar previamente cadastrados.

Fluxo principal: (1) o aluno informa os dois exercícios do bi-set, com suas respectivas repetições e cargas; (2) o sistema valida que há exatamente dois exercícios associados ao mesmo `grupoSequencia`; (3) o sistema persiste as séries com `tipoTecnica = BI_SET`; (4) o sistema retorna o treino registrado, incluindo o 1RM estimado de cada exercício.

Fluxo alternativo: Caso apenas um exercício seja informado para um bi-set, o sistema rejeita o registro e retorna erro de validação (RNF12).

12 Arquitetura da Aplicação

O RepMax adota a arquitetura em camadas (Controller, Service, Repository, Entity), amplamente utilizada em aplicações Spring Boot, complementada pelo uso de DTOs.

- **Controller:** recebe as requisições HTTP, realiza validação básica e delega à camada de Service, retornando DTOs de resposta.
- **Service:** concentra as regras de negócio — cálculo de 1RM, validação de técnicas combinadas (bi-set/tri-set/monster set/drop set), regras de vínculo aluno-pessoal.

- **Repository:** acessa o PostgreSQL via Spring Data JPA, com consultas personalizadas para histórico e progressão.
- **Entity:** representa as tabelas do banco descritas na Seção 9, mapeando os relacionamentos com anotações JPA (`@OneToMany`, `@ManyToOne`, `@OneToOne`).
- **DTO:** evita a exposição direta das entidades, com exemplos como `TreinoRequestDTO`, `TreinoResponseDTO`, `SerieExecutadaDTO`, `ProgressoDTO` e `PlanoTreinoDTO`.

13 Endpoints da API (Principais)

Método	Endpoint	Descrição
POST	/api/usuarios	Cadastra um novo usuário.
POST	/api/auth/login	Autentica um usuário.
GET/POST	/api/exercicios	Lista ou cadastra exercícios.
GET	/api/exercicios/{id}/instrucoes	Retorna as instruções de execução de um exercício.
POST	/api/treinos	Registra um novo treino.
GET	/api/alunos/{id}/treinos	Lista o histórico de treinos de um aluno.
GET	/api/alunos/{id}/progresso/{exercicioId}	Retorna a evolução de carga/volume de um exercício.
POST	/api/rotinas	Cria uma rotina de treino.
DELETE	/api/vinculos/{id}	Encerra um vínculo entre aluno e personal.
POST	/api/treinos/{id}/feedback	Aluno registra feedback de um treino.

14 Regras de Negócio

14.1 Não negatividade da carga

Regra de negócio

A carga registrada em uma série não pode ser negativa; o número de repetições deve ser maior que zero.

14.2 Enumeração de bi-set, tri-set e monster-set

Regra de negócio

Um bi-set deve conter exatamente dois exercícios distintos no mesmo **grupoSequencia**; um tri-set, exatamente três; um monster set, quatro ou mais.

14.3 Diminuição da carga em drop set

Regra de negócio

Em um drop set, a carga de cada série subsequente deve ser estritamente menor que a da série anterior dentro do mesmo exercício.

14.4 Fórmula do cálculo de 1RM e restrição a quantidade de repetições

Regra de negócio

Em cálculo de 1RM deve utilizar a fórmula de Epley e ser restrito a séries com até 10–12 repetições, faixa em que a fórmula mantém boa precisão.

14.5 Um aluno não pode visualizar treinos e histórico de outro aluno

Regra de negócio

Um aluno não pode acessar o histórico ou os planos de treino de outro aluno, salvo o personal, que acessa apenas os dados dos alunos a ele vinculados.

14.6 Transparência no propósito de feedbacks do Rep Max

Regra de negócio

Alertas e sugestões do sistema (ex.: progresso estagnado) têm caráter informativo e não substituem a orientação de um profissional de educação física.

15 Segurança e Privacidade

- **Autenticação:** obrigatória para acesso a qualquer recurso protegido da API (JWT).
- **Autorização por perfil (RBAC):** aluno, personal trainer e administrador possuem permissões distintas.
- **Criptografia de senhas:** armazenamento via Argon2.
- **Validação de entrada:** todos os dados recebidos devem ser validados antes do processamento.
- **Proteção de dados entre usuários:** um aluno só acessa seus próprios dados; um personal só acessa dados de alunos vinculados e aceitos.
- **HTTPS em produção:** toda comunicação deve ocorrer de forma criptografada em um eventual ambiente de produção.

16 Conclusão

O sistema RepMax, apresentado ao longo desta documentação, propõe uma solução estruturada para o registro e acompanhamento de treinos de musculação, contemplando desde funcionalidades básicas de um log book até técnicas avançadas de intensidade (bi-set, tri-set, monster set e drop set) e um modelo colaborativo entre aluno e personal trainer. O projeto foi concebido de forma incremental, iniciando em um escopo simples e evoluindo em fases, o que permite consolidar conceitos de Java, Spring Boot e persistência com JPA/PostgreSQL ao longo do desenvolvimento.

Reforça-se que, no momento desta entrega, nenhuma das funcionalidades apresentadas foi implementada: este documento reflete integralmente a fase de análise e especificação do projeto, servindo de base para as etapas subsequentes de implementação.

17 Referências

- SOCIEDADE BRASILEIRA DE ENDOCRINOLOGIA E METABOLOGIA (SBEM). *Dados sobre a prática de atividade física no Brasil*. [link oficial e ano — campo editável].
- ORACLE CORPORATION. *Java Documentation*. [link oficial — campo editável].
- VMWARE / SPRING TEAM. *Spring Boot Reference Documentation*. [link oficial — campo editável].
- VMWARE / SPRING TEAM. *Spring Data JPA — Reference Documentation*. [link oficial — campo editável].
- VMWARE / SPRING TEAM. *Spring Security Reference Documentation*. [link oficial — campo editável].
- THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. [link oficial — campo editável].
- OBJECT MANAGEMENT GROUP (OMG). *Unified Modeling Language (UML) Specification*. [versão e link — campo editável].
- EPLEY, B. *Poundage Chart*. Boyd Epley Workout, 1985. [referência da fórmula de 1RM — campo editável].
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: uma abordagem profissional*. [edição e ano — campo editável].